
BLM0121 NESNEYE YÖNELİK PROGRAMLAMA DERSİ

PROJE RAPORU

Öğrenci Adı: Kürşat

Öğrenci Soyadı: Şahin

Öğrenci Numarası: 25360859432

Sorumlu Eğitim Görevlileri: Doç. Dr. Ergün GÜMÜŞ

Arş. Gör. Şeyma DOĞRU

Arş. Gör. Zeynep BARUT

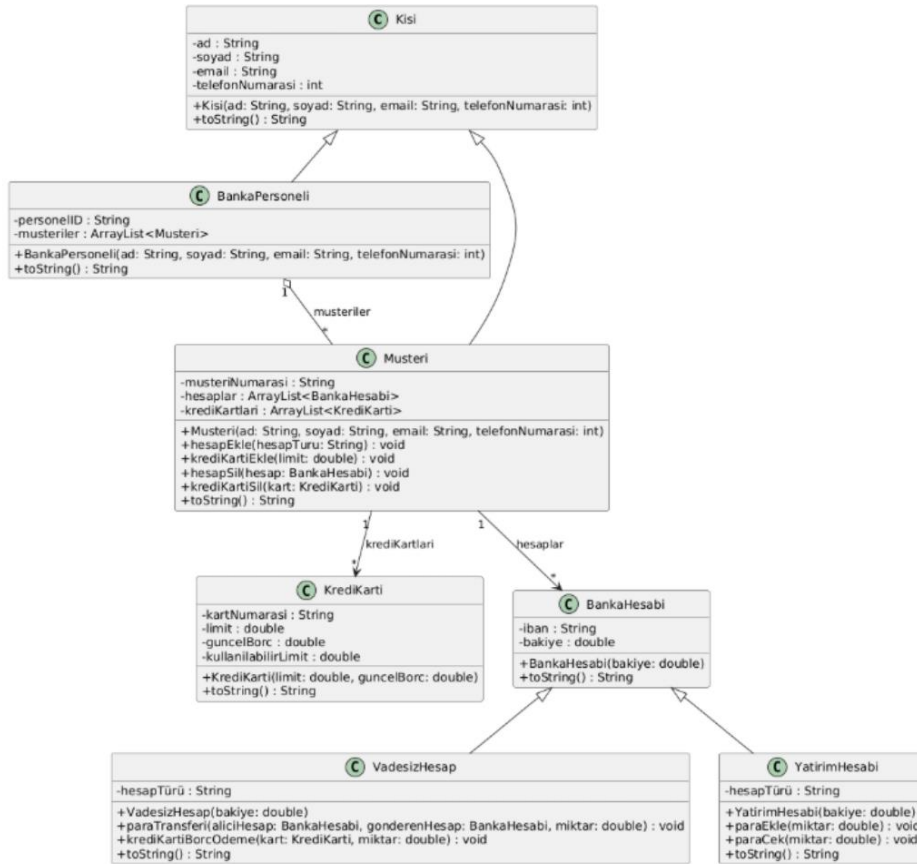
Proje Kaynak Kodları: [GitHub](#)

Proje Anlatım Videosu: [YouTube](#)

Proje Ön Bilgilendirmesi: Gerçekleştirilen projede bir banka simülasyonu gerçekleştirilmektedir. Bu simülasyonda nesneye yönelik programlamaya uygun şekilde sınıflar oluşturularak, sınıf ve sınıflar arası etkileşimlerin kavranması ve pekiştirilmesi amaçlanmaktadır.

Proje Senaryosu: Gerçekleştirilen bu proje için yapılan kodlamalar bir senaryoya uygun şekilde yapılmaktadır. Bu senaryoya göre; bir banka personeli yalnızca sistemin kendi içerisinde oluşturulabilir, her müşteriyle ilgilenen bir banka personeli olmalıdır, kullanıcı için hesap açma ve silme, kredi kartı oluşturma ve silme gibi işlemler yalnızca banka personeli tarafından gerçekleştirilebilmektedir. Müşteri ise kendisine tanımlanan hesaplar ve kredi kartları ile ilgili işlemler gerçekleştirebilmektedir.

PROJE İÇİN TANIMLANAN UML DİYAGRAMI



(Görsel 1.1)

- Proje için tanımlanan UML Diyagramı yukarıdaki Görsel 1.1’de gösterilmektedir. Buradaki UML Diyagramında istenilen sınıf ve metotların dışında, belirlenen senaryo gereği ekstra olarak metotlar eklenmiştir. Görsel 1.1’de paylaşılan UML Diyagramında olmayan fakat proje dokümantasyonunda istenilen ‘BankaService’ sınıfı üzerinden işlemler gerçekleştirilerek oluşturulan banka sisteminin çalışması sağlanmaktadır.

Gerçekleştirilen projenin ilk aşaması olarak bir banka personeli oluşturulmaktadır. Oluşturulan ilk banka personeli ‘patron’ olarak tanımlanmaktadır. Oluşturulan banka personeli nesnesinin doğrudan kod üzerinden oluşturulmasının sebebi proje senaryosunda da anlatıldığı gibi banka personelinin yalnızca yetkili kişiler tarafından oluşturulmasından kaynaklanmaktadır.

Yukarıda anlatılmakta olan işlemin Main.Java ve BankaService.Java sınıfları içerisindeki kod örnekleri aşağıdaki gibidir.

```
public class BankaService {  
    /*  
     * Burada yeni müşteri oluşturulurken hem Musteri sınıfı içerisindeki konstrüktöre veriler gönderilerek  
     * nesne oluşturuluyor. Hem de random sınıfa ile yeni bir müşteri numarası oluşturuluyor. En son da işlemler  
     * devam edebilsin diye bir müşteri nesnesi dışarıya döndürülüyor.  
     * Metotların static olmasının sebebi burada sadece işlem yapılacak. Herhangi bir BankaService sınıfından  
     * nesneye ihtiyaç yoktur.  
     */  
    public static BankaPersoneli bankaPersonelOlusturma(String ad, String soyad, String email, int tel) {  
        //PERSONEL ID'NİN RASTGELE OLUŞTURULABİLMESİ İÇİN RANDOM NESNESİ OLUŞTURMA  
        Random r = new Random();  
  
        String personelID = "";  
        //PARAMETRE OLARAK ALINAN BİLGİLERİN BankaPersoneli NESNESİNE AKTARILARAK YENİ PERSONEL OLUŞTURULMASI  
        BankaPersoneli yeniPersonel = new BankaPersoneli(ad,soyad,email,tel);  
  
        for (int i = 0 ; i < 10 ; i++) {  
            personelID += (r.nextInt(9) + 1);  
        }  
        //RASTGELE OLUŞTURULAN ID'NİN PERSONELE TANIMLANMASI  
        yeniPersonel.setPersonelID(personelID);  
        System.out.println("Yeni Banka Personeli Oluşturuldu!");  
        //SİSTEMDEKİ TÜM KİŞİLERİN BİR ARADA TUTULMASI İÇİN ARRAYLIST İÇERİSİNE KAYDEDİLMESİNİ SAĞLAYAN METOT  
        Kisi.kisiEkle(yeniPersonel);  
        return yeniPersonel;  
    }  
}  
  
//İŞLEMLERİN YAPILABİLMESİ İÇİN BANKA PERSONELİNE İHTİYAÇ VAR  
BankaPersoneli patron = BankaService.bankaPersonelOlusturma("Kürşat", "Şahin", "kursat@gmail.com", 1234567);
```

Gerçekleştirilen proje bir veri tabanına bağlı olmadığı için kullanıcı işlemlerini gösterebilmek adına kod üzerinden bir müşteri nesnesi oluşturulmaktadır. Bu sayede kodun ileriki kısımlarında kullanıcı ile ilgili işlemlerin gösterilmesi amaçlanmaktadır.

```
//İŞLEMLERİN GÖSTERİLEBİLMESİ İÇİN İLK MÜŞTERİNİN OLUŞTURULMASI  
Musteri ilkMusteri = BankaService.musteriOlusturma("Halit", "Kılıç", "halit@gmail.com", 12345678, patron);
```

```
public static Musteri musteriolusturma(String ad, String soyad, String email, int tel, BankaPersoneli personel) {  
    //Kullanıcıdan başlangıçta bu bilgileri alarak Musteri nesnesi oluşturacağız.  
    //BANKA PERSONELİNDE OLDUĞU GİBİ RASTGELE MÜŞTERİ NUMARASI OLUŞTURMAK İÇİN OLUŞTURULAN RANDOM SINIFINA AİT NESNE  
    Random r = new Random();  
    String musterino = "";  
    //ALINAN BİLGİLERİN MÜŞTERİ SINIFINA AKTARILARAK NESNE OLUŞTURULMASI  
    Musteri yeniMusteri = new Musteri(ad,soyad,email,tel);  
  
    //Müşteri numarasını 11 haneli kabul ettim.  
    for (int i = 0 ; i < 10 ; i++) {  
        musterino += (r.nextInt(9)+ 1);  
    }  
    //RASTGELE OLUŞTURULAN MÜŞTERİ NUMARASININ MÜŞTERİYE TANIMLANMASI  
    yeniMusteri.setMusteriNumarasi(musterino);  
    System.out.println("Müşteri Kaydı Başarıyla Gerçekleşti!");  
    Kisi.kisiEkle(yeniMusteri);  
    //OLUŞTURULAN MÜŞTERİNİN PERSONEL İLE İLİŞKİLENDİRİLMESİ  
    personel.musteriEkle(yeniMusteri);  
    return yeniMusteri;  
}
```

Kullanıcı işlemlerinin gerçekleştirilebilmesi için banka hesabı ve kredi kartlarının oluşturulması ve kredi kartı borcunun güncellenmesi.

```
BankaService.musteriHesapEkle(ilkMusteri, "Vadesiz Hesap");  
BankaService.musteriHesapEkle(ilkMusteri, "Yatırım Hesabı");  
BankaService.krediKartiEkle(ilkMusteri, 15000);  
ilkMusteri.getKrediKartlari().get(0).setGuncelBorc(1453);
```

Proje içerisinde kullanılan sınıf ve metot sayısı fazla olduğundan kaynaklı her sınıfın ve sınıf içerisindeki metodun ne yaptığı ve nasıl çalıştığı paylaşamamaktadır. Bunun için giriş sayfasında paylaşılan [GitHub](#) linki kullanılarak kaynak kodlara ulaşılabilmektedir.

Konsol üzerinden kullanıcı adı ve şifre alınarak sisteme giriş yapılması sağlanmaktadır. Gerçekleştirilen bu projede kullanıcı adı email, şifre ise telefon numarası olarak kabul edilmektedir.

```
//BURADAKİ MAİL VE TELEFON NUMARASI KULLANICI ADI ŞİFRE GİBİ DÜŞÜNÜLEBİLİR
//PROGRAMI KULLANAN KİŞİDEN BİR KULLANICI ADI VE ŞİFRE GİRMESİ İSTENECEK
Kisi kisiOrnek = null;
System.out.print("Kullanıcı Adınızı Giriniz: ");
String kullanıcıAdi =input.nextLine();
System.out.print("Şifre Giriniz: ");
int sifre = input.nextInt();
boolean kullanıcıVarMi = false;
//KULLANICIDAN ALINAN VERİLERİN BİR KULLANICI İLE EŞLEŞİP EŞLEŞMEDİĞİNİ KONTROL EDECEĞİZ
ArrayList<Kisi> kayitliKisiler = Kisi.getKisiler();
for (Kisi k : kayitliKisiler) {

    if (k.getEmail().equals(kullanıcıAdi) && k.getTelefonNumarasi() == sifre) {
        //GİRİŞ BAŞARILI İSE K NESNESİNDEKİ BİLGİLERİ kisiOrnek NESNESİNE ATIYORUZ
        kisiOrnek = k;
        kullanıcıVarMi= true;
        System.out.println("Giriş Başarılı!");
        break;
    }
}

if (!kullanıcıVarMi) {
    System.out.println("Kullanıcı Bulunamadı!");
}
```

Paylaşılan kod örneğinde kullanıcıdan kullanıcı adı ve şifre alınmakta, bu alınan girdiler sistemde kayıtlı olan tüm kişiler üzerinden aranmaktadır. Kullanıcı sistemde kayıtlı ise eşleşen bilgiler oluşturulan Kisi sınıfından kisiOrnek nesnesine aktarılmaktadır. Girilen bilgiler kayıtlı hiçbir kullanıcı ile eşleşmez ise “Kullanıcı Bulunamadı” mesajı ekrana yazdırılacaktır.

```
Kullanıcı Adınızı Giriniz: kursat@gmail.com
Şifre Giriniz: 1234567
Giriş Başarılı!
```

```
Kullanıcı Adınızı Giriniz: kursat@gmail.com
Şifre Giriniz: 12345678
Kullanıcı Bulunamadı!
```

Başarılı giriş yapan kullanıcının müşteri veya banka personeli olup olmadığı 'instanceof' ile kontrol edilerek hangi işlemlerin yapılabileceği konsol üzerinden yazdırılmaktadır. Giriş yapan kullanıcı banka personeli ise yapabileceği işlemler *personelIslemler()* metodu ile gösterilmektedir ve kendisinin sorumlu olduğu banka müşterilerinin listesi BankaPersoneli sınıfında tanımla metot ile tanımlanmaktadır.

```
public static void personelIslemler() {  
    System.out.println("HANGİ İŞLEMİ YAPMAK İSTİYORSUNUZ?");  
    System.out.println("1) Müşteri Kaydı Oluşturma");  
    System.out.println("2) Müşteri Hesap Ekleme");  
    System.out.println("3) Müşteri Hesap Silme");  
    System.out.println("4) Müşteri Kredi Kartı Ekle");  
    System.out.println("5) Müşteri Kredi Kartı Sil");  
    System.out.println("6) Güncel Müşterileri Listele");  
    System.out.println("7) Müşteri Seçimi");  
    System.out.println("8) Çıkış");  
}
```

Başarılı giriş yapan kullanıcının müşteri olduğu durumda ise yapabileceği işlemler farklı olduğundan bu işlemlerin listelenmesi için başka bir metot tanımlanmıştır. Bu işlemler *musteriIslemler()* metodu ile gösterilmektedir.

```
public static void musterisiIslemler() {  
    System.out.println("HANGİ İŞLEMİ YAPMAK İSTİYORSUNUZ?");  
    System.out.println("1) Hesap Bilgilerini Göster");  
    System.out.println("2) Hesaba Para Yatır");  
    System.out.println("3) Hesaptan Para Çek");  
    System.out.println("4) Hesaplar Arası Para Transferi");  
    System.out.println("5) Kredi Kartı Borç Ödeme");  
    System.out.println("6) Çıkış");  
}
```

Buradaki işlem bilgilendirilmesi yapıldıktan sonra switch-case yapısı ile çalışılarak yapılacak işlemler gerçekleştirilmektedir. Banka personelinin yapacağı işlemler listelendikten sonra yapmak istediği işlemin seçilmesi istenmektedir.

```
if (kisiOrnek instanceof BankaPersoneli) {  
    //GİRİŞ YAPAN KİŞİ BANKA PERSONELİ İSE kisiOrnek NESNESİ BANKA PERSONELİ SINIFINA CAST EDİLİR  
    BankaPersoneli personel = (BankaPersoneli) kisiOrnek;  
    //BANKA PERSONELİNE BAĞLI MÜŞTERİLERİN GETİRİLMESİ  
    ArrayList<Musteri> musteriler = personel.getMusteriler();  
  
    //MÜŞTERİ SEÇİMİ İÇİN GEREKLİ METOTUN ÇAĞIRILMASI  
    Musteri secilenMusteri = BankaService.musteriSecimi(musteriler);  
}
```

```
public static Musteri musterisecimi(ArrayList<Musteri> musteriler) {  
    Scanner input = new Scanner(System.in);  
    //PERSONELİN HATALI GİRİŞ YAPMASINA KARŞIN MÜŞTERİ SAYISININ ALINMASI  
    int musterisayisi = musteriler.size();  
    int musterisecimi;  
    //BOŞ BİR MÜŞTERİ NESNESİNİN OLUŞTURULMASI  
    Musteri secilenMusteri = null;  
    //GİRİŞ YAPAN PERSONELE BAĞLI MÜŞTERİLERİN LİSTELENMESİ  
    for (int i = 0 ; i < musterisayisi ; i++) {  
        System.out.printf("%d) " + musteriler.get(i).toString(),i+1);  
    }  
    //MÜŞTERİ SAYISI 1 VEYA DAHA FAZLA İSE SEÇİM İŞLEMİNİN YAPILMASI  
    if (musterisayisi >= 1) {  
        //PERSONELİN DOĞRU BİR SEÇİM YAPANA KADAR SİSTEMDE KALMASINI SAĞLAYAN DÖNGÜ  
        while(true) {  
            System.out.print("Hangi Müşteriniz ile İşlem Yapmak İstiyorsunuz: ");  
            musterisecimi = input.nextInt();  
            if (musterisecimi > musterisayisi || musterisecimi < 0) {  
                System.out.println("Hatalı Seçim Yaptınız Lütfen Tekrar Seçim Yapınız!");  
            }  
            else {  
                break;  
            }  
        }  
        //BELİRLENEN MÜŞTERİ NESNESİNİN BİLGİLERİNİN secilenMusteri'YE AKTARILMASI  
        secilenMusteri = musteriler.get(musterisecimi-1);  
    }  
    //MÜŞTERİNİN GERİ DÖNDÜRÜLMESİ  
    return secilenMusteri;  
}
```

Banka personelinin başarılı girişi sonucu başlangıçta kendisinin sorumluluğu altında olan müşteriler ve müşteri bilgileri listelenmektedir.

```
Kullanıcı Adınızı Giriniz: kursat@gmail.com  
Şifre Giriniz: 1234567  
Giriş Başarılı!  
1) Müşteri Numarası: 3695533351 Halit  
İban: TR52874626837977765853358, Bakiye: 0.0, Hesap Türü: Vadesiz Hesap  
İban: TR37571866897594346779837, Bakiye: 0.0, Hesap Türü: Yatırım Hesabı  
Kart Numarası: 179965211254293, Limit: 15000.0, Güncel Borç: 1453.0, Kullanılabilir Limit: 13547.0
```

Yapılacak işlemler ve personele bağlı müşteriler listelendikten sonra hangi müşteri için işlem yapacağı bilgisi istenmektedir. Sırasıyla banka personelinin yapabileceği işlemlerin kodu ve çıktısı aşağıda paylaşılan görsellerde gösterilmektedir.

```
Hangi Müşteriniz ile İşlem Yapmak İstiyorsunuz: 1
HANGİ İŞLEMİ YAPMAK İSTİYORSUNUZ?
1) Müşteri Kaydı Oluşturma
2) Müşteri Hesap Ekleme
3) Müşteri Hesap Silme
4) Müşteri Kredi Kartı Ekle
5) Müşteri Kredi Kartı Sil
6) Güncel Müşterileri Listele
7) Müşteri Seçimi
8) Çıkış
Yapılacak İşlemi Seçin:
```

```
System.out.print("Yapılacak İşlemi Seçin: ");
int secilenIslem = input.nextInt();
input.nextLine();

switch(secilenIslem) {
    case 1://MÜŞTERİ KAYDININ OLUŞTURULMASI İÇİN GEREKLİ BİLGİLERİN ALINMASI
        System.out.print("Müşteri Adını Giriniz: ");
        String musterAdi = input.nextLine();
        System.out.print("Müşteri Soyadını Giriniz: ");
        String musterSoyadi = input.nextLine();
        System.out.print("Müşteri E-Mail Bilgisini Giriniz: ");
        String musterEmail = input.nextLine();
        System.out.print("Müşteri Telefon Numarasını Giriniz: ");
        int tel = input.nextInt();
        BankaService.musteriOlusturma(musterAdi, musterSoyadi, musterEmail, tel, personel);
        satirBosluguEkleme();
        break;
```

```
Yapılacak İşlemi Seçin: 1
Müşteri Adını Giriniz: Elif Sude
Müşteri Soyadını Giriniz: Telli
Müşteri E-Mail Bilgisini Giriniz: elif@gmail.com
Müşteri Telefon Numarasını Giriniz: 123456789
Müşteri Kaydı Başarıyla Gerçekleşti!
```

Yukarıdaki kod ve kodun çıktısı paylaşılan görselde banka personeli bir müşteri nesnesi oluşturulması için gerekli bilgiler alınmış ve müşteri nesnesi oluşturulmuştur. Oluşturulan müşteri nesnesi, oluşturan personel ile ilişkilendirilmiştir.

Banka personelinin müşteri ile ilgili yapacağı ikinci işlem ise müşteri adına hesap açmaktır. Banka personeli bu aşamada “Vadesiz Hesap” veya “Yatırım Hesabı” oluşturabilmektedir. İstenilen girdiye yukarıda belirtilen formattaki iki girdiden farklı bir girdi girişi olursa hata mesajı ekrana yazdırılmaktadır. İstenilen formatta giriş olduğunda ise müşteri adına girdi yapılan hesap oluşturulmaktadır. Bu aşamada banka personelinin altıncı işlemi olan “Güncel Müşterileri Listele” seçeneği de kullanılacaktır. Böylelikle metodun çalışıp çalışmadığı da görülmüş olacaktır.

Banka hesabı oluşturulmadan önceki kullanıcı bilgilerinin yazdırılması:

```
2) Müşteri Numarası: 8756984744 Elif Sude
```

Müşteri adına hesap oluşturma işleminin kaynak kodu:

```
case 2://MÜŞTERİ ADINA HESAP AÇILMA İŞLEMİ
System.out.print("Açmak İstedığınız Hesap Türünü Yazınız: (Vadesiz Hesap/ Yatırım Hesabı)");
String acilacakHesap = input.nextLine();
if (acilacakHesap.equals("Vadesiz Hesap") || acilacakHesap.equals("Yatırım Hesabı")) {
    BankaService.musteriHesapEkle(secilenMusteri, acilacakHesap);
    satirBosluguEkleme();
}
else {
    System.out.println("İstenilen Formatta Hesap Türü Seçiniz!");
}
break;
```

Hesap oluşturma aşamasında oluşan kod çıktısı:

```
Açmak İsteddiğiniz Hesap Türünü Yazınız: (Vadesiz Hesap/ Yatırım Hesabı)Vadesiz Hesap
Vadesiz Hesabınız oluşturuldu!
```

Müşteri bilgilerinin güncel şekilde gösterilmesi:

```
2) Müşteri Numarası: 8756984744 Elif Sude
İban: TR51912124457593168952346, Bakiye: 0.0, Hesap Türü: Vadesiz Hesap
```

Hatalı girdi durumunda oluşan çıktı:

```
Açmak İsteddiğiniz Hesap Türünü Yazınız: (Vadesiz Hesap/ Yatırım Hesabı)VadesizHesap
İstenilen Formatta Hesap Türü Seçiniz!
```

Banka personelinin üçüncü olarak yapabildiği işlem, ilgilenmekte olduğu müşterinin banka hesaplarını silmektir. Bu aşamada BankaService.musteriHesapSil() metodunda gerekli kontroller yapılarak müşteri hesabının silinmesi sağlanmaktadır. Eğer müşterinin silinmek istenen hesabında parası bulunuyorsa silme işlemi gerçekleşmemektedir.

Banka hesabı silme işleminin kaynak kodu:

```
case 3://MÜŞTERİNİN BANKA HESABININ SİLİNMESİ
    if (secilenMusteri.getHesaplar().size() > 0) {
        BankaService.musteriHesapListele(secilenMusteri);
        System.out.print("Hangi Hesabı Silmek İstersiniz: ");
        int secilenHesapIndex = input.nextInt();
        //MÜŞTERİNİN HESAPLARINA ERIŞİLEREK GİRİLEN İNDEX BİLGİSİNDEKİ HESABI GETİRME İŞLEMİ
        //secilenHesapIndex - 1 DENMESİNİN SEBEBİ İNDEXLEMENİN 0 DAN BAŞLIYOR OLMASIDIR
        BankaHesabi secilenHesap = secilenMusteri.getHesaplar().get(secilenHesapIndex -1);
        BankaService.musteriHesapSil(secilenMusteri, secilenHesap);
        satirBosluguEkleme();
    }
    else {
        System.out.println("Müşterinin Hesabı Bulunmamaktadır!");
        satirBosluguEkleme();
    }

    break;
```

BankaService.musteriHesapSil() metodunun kaynak kodu:

```
public static void musterihesapSil(Musteri muster, BankaHesabi hesap) {
    //MÜŞTERİNİN HESAPLARI ARRAYLIST OLARAK ALINIR
    ArrayList<BankaHesabi> hesaplar = muster.getHesaplar();
    int hesapIndexi = hesaplar.indexOf(hesap);

    //MÜŞTERİNİN BAKİYESİNİN 0 OLUP OLMADIĞI KONTROL EDİLİR
    if (hesaplar.get(hesapIndexi).getBakiye() != 0) {
        System.out.println("Lütfen Öncelikle Bakiyenizi Başka Hesaba Aktarın");
    }
    else {
        muster.hesapSil(hesap);
        System.out.println("Hesabınız Başarıyla Silindi");
    }
}
```

musteri.hesapSil() metodunun kaynak kodu:

```
//ArrayList<BankaHesabi> hesaplar = new ArrayList<>(); TANIMLAMASI YAPILMIŞTIR  
public void hesapSil(BankaHesabi hesap) {  
    this.hesaplar.remove(hesap);  
}
```

Hesap silme işleminin başarılı sonuçlanması ile oluşan konsol çıktısı:

```
1) Iban: TR51912124457593168952346, Bakiye: 0.0, Hesap Türü: Vadesiz Hesap  
Hangi Hesabı Silmek İstersiniz: 1  
Hesabınız Başarıyla Silindi
```

Banka personelinin yapabileceği dört ve beşinci işlemler kredi kartı ile ilgili olacaktır. İlk olarak müşteri için kredi kartı oluşturabilir ve gerekli şartlar sağlandığında bu kredi kartını silebilmektedir. Kredi kartının silinebilmesi için müşterinin silinmek istenen kredi kartında hiçbir borç bulunmaması gerekmektedir.

Kredi kartı oluşturma işleminin kaynak kodu:

```
case 4://MÜŞTERİYE KREDİ KARTI TANIMLANMASI  
    System.out.print("Kredi Kartı İçin Limit Belirleyin: ");  
    int limit = input.nextInt();  
    BankaService.krediKartiEkle(secilenMusteri, limit);  
    satirBosluguEkleme();  
    break;
```

Kredi kartı oluşturulmadan önce kullanıcı bilgileri:

```
Yapılacak İşlemi Seçin: 0  
1) Müşteri Numarası: 9158934586 Halit  
Iban: TR88732824766343349475554, Bakiye: 0.0, Hesap Türü: Vadesiz Hesap  
Iban: TR58337522678457113961999, Bakiye: 0.0, Hesap Türü: Yatırım Hesabı  
Kart Numarası: 844335274617292, Limit: 15000.0, Güncel Borç: 1453.0, Kullanılabilir Limit: 13547.0  
Kart Numarası: 142655844178378, Limit: 3000.0, Güncel Borç: 0.0, Kullanılabilir Limit: 3000.0
```

Kredi kartı oluşturma aşaması:

```
Yapılacak İşlemi Seçin: 4  
Kredi Kartı İçin Limit Belirleyin: 5000  
Kredi Kartınız Başarıyla Tanımlandı!
```

Kredi kartı oluşturulduktan sonra kullanıcı bilgileri:

```
1) Müşteri Numarası: 9158934586 Halit  
Iban: TR88732824766343349475554, Bakiye: 0.0, Hesap Türü: Vadesiz Hesap  
Iban: TR58337522678457113961999, Bakiye: 0.0, Hesap Türü: Yatırım Hesabı  
Kart Numarası: 844335274617292, Limit: 15000.0, Güncel Borç: 1453.0, Kullanılabilir Limit: 13547.0  
Kart Numarası: 142655844178378, Limit: 3000.0, Güncel Borç: 0.0, Kullanılabilir Limit: 3000.0  
Kart Numarası: 727869468457754, Limit: 5000.0, Güncel Borç: 0.0, Kullanılabilir Limit: 5000.0
```

Kredi kartı silme işleminin kaynak kodu:

```
case 5://MÜŞTERİ KREDİ KARTI SİLME İŞLEMİ  
if (secilenMusteri.getKrediKartlari().size() > 0) {  
    BankaService.musteriKrediKartiListele(secilenMusteri);  
    System.out.print("Hangi Kartı Silmek İstersiniz: ");  
    int secilenKartIndex = input.nextInt();  
    //GİRİLEN İNDEX BİLGİSİNE GÖRE KREDİ KARTI NESNESİNİN ÇEKİLMESİ  
    KrediKarti secilenKart = secilenMusteri.getKrediKartlari().get(secilenKartIndex - 1 );  
    BankaService.krediKartiSil(secilenMusteri, secilenKart);  
    satirBosluguEkleme();  
}  
else {  
    System.out.println("Müşterinin Kredi Kartı Bulunmamaktadır!");  
    satirBosluguEkleme();  
}  
  
break;
```

BankaService.krediKartiSil() metodunun kaynak kodu:

```
public static void krediKartiSil(Musteri muster, KrediKarti kart) {  
  
    //MÜŞTERİNİN SEÇİLEN KARTINDAKİ GÜNCEL BORÇ BİLGİSİNİN ÇEKİLMESİ  
    double guncelBorc = kart.getGuncelBorc();  
    //GÜNCEL BORÇ 0 İSE KART SİLİNEBİLİR  
    if (guncelBorc == 0) {  
        muster.krediKartiSil(kart);  
        System.out.println("Kredi Kartınız Başarıyla Silindi");  
    }  
    else { // GÜNCEL BORÇ 0 DEĞİL İSE MÜŞTERİ BORÇ ÖDEMESİ YAPMALI  
        System.out.println("Kartınızı Silmeden Önce Borç Ödemesi Yapmalısınız!");  
        System.out.println("Güncel Borcunuz: " + guncelBorc);  
    }  
}
```

Borcu olan kartı silmeye çalışılması durumunda oluşacak kod çıktısı:

```
1) Kart Numarası: 844335274617292, Limit: 15000.0, Güncel Borç: 1453.0, Kullanılabilir Limit: 13547.0  
2) Kart Numarası: 142655844178378, Limit: 3000.0, Güncel Borç: 0.0, Kullanılabilir Limit: 3000.0  
3) Kart Numarası: 727869468457754, Limit: 5000.0, Güncel Borç: 0.0, Kullanılabilir Limit: 5000.0  
Hangi Kartı Silmek İstersiniz: 1  
Kartınızı Silmeden Önce Borç Ödemesi Yapmalısınız!  
Güncel Borcunuz: 1453.0
```

Borcu olmayan kartın silinmesi durumunda oluşacak çıktı:

```
1) Kart Numarası: 844335274617292, Limit: 15000.0, Güncel Borç: 1453.0, Kullanılabilir Limit: 13547.0
2) Kart Numarası: 142655844178378, Limit: 3000.0, Güncel Borç: 0.0, Kullanılabilir Limit: 3000.0
3) Kart Numarası: 727869468457754, Limit: 5000.0, Güncel Borç: 0.0, Kullanılabilir Limit: 5000.0
Hangi Kartı Silmek İstersiniz: 2
Kredi Kartınız Başarıyla Silindi
```

```
1) Müşteri Numarası: 9158934586 Halit
İban: TR88732824766343349475554, Bakiye: 0.0, Hesap Türü: Vadesiz Hesap
İban: TR58337522678457113961999, Bakiye: 0.0, Hesap Türü: Yatırım Hesabı
Kart Numarası: 844335274617292, Limit: 15000.0, Güncel Borç: 1453.0, Kullanılabilir Limit: 13547.0
Kart Numarası: 727869468457754, Limit: 5000.0, Güncel Borç: 0.0, Kullanılabilir Limit: 5000.0
```

Müşterinin borcu olması durumunda borç ödemesi müşteri tarafından yapılacaktır. Bu işlem müşterinin yapabileceği işlemler kısmına geçildiğinde gösterilecektir. Raporun dokuzuncu sayfasında belirtilen “**Güncel Müşterileri Listele**” metodunun kaynak kodu aşağıdaki gibidir:

```
case 6://PERSONELE AİT MÜŞTERİLER LİSTEYE ATANARAK BİLGİLERİYLE YAZDIRILMAKTADIR
ArrayList<Musteri> guncelMusteriler = personel.getMusteriler();
for (int i = 0 ; i < guncelMusteriler.size() ; i++) {
    System.out.printf("%d) " + guncelMusteriler.get(i).toString(),i+1);
    satirBosluguEkleme();
}
break;
```

Bununla birlikte programa bir kez giriş yapan banka personelin bir başka müşterisiyle ilgilenmek istediğinde sürekli programı kapatıp açmak zorunda kalmaması gerekmektedir. Bu sebeple banka personeli için “Müşteri Seçimi” seçeneği tanımlanmıştır. Bu seçenek seçildiğinde programın en başında yapılan müşteri seçimi sekmesine geri dönülerek seçilen müşteri değiştirilir ve işlemler yeni seçilen müşteri üzerinden yapılmaya devam edilmektedir.

Müşteri değiştirme işleminin kaynak kodu:

```
case 7:
    secilenMusteri = BankaService.musteriSecimi(personel.getMusteriler());
    satirBosluguEkleme();
    break;
```

BankaService.musteriSecimi() metodu raporun önceki kısımlarında gösterilmiştir.

Yukarıdaki sayfalarda banka personeli için tanımlanan işlemler açıklanmaktadır. Müşteri için tanımlanan işlemler raporun kalan kısmında gösterilmektedir. İlk olarak kullanıcı giriş yaptıktan sonra yapabileceği işlemler *musteriIslemler()* metodu içerisinde listelenmektedir.

Müşteriye ait hesap ve kredi kartı bilgilerinin listelenmesinin kaynak kodu:

```
case 1: //MÜŞTERİNİN BİLGİLERİNİN YAZDIRILMASI
    System.out.println(musteri.toString());
    satirBosluguEkleme();
    break;
```

```
Yapılacak İşlemi Seçin: 1
Müşteri Numarası: 4733278967 Halit
İban: TR87658956195332874517295, Bakiye: 0.0, Hesap Türü: Vadesiz Hesap
İban: TR11449752144681515529544, Bakiye: 0.0, Hesap Türü: Yatırım Hesabı
Kart Numarası: 798336937877454, Limit: 15000.0, Güncel Borç: 1453.0, Kullanılabilir Limit: 13547.0
```

Hesap bilgileri ekrana yazdırıldıktan sonra bir sonraki yapacağı işlem banka hesaplarına para yatırmaktır. Burada kullanıcının hesap bilgileri listelenir ve hangi hesaba para yatırmak istendiği sorulur. Daha sonrasında girilen girdi, gerekli hata kontrollerinden geçirilerek hesaba para eklenmesi sağlanmaktadır.

Müşterinin hesaba para yatırma işleminin kaynak kodu:

```
case 2://MÜŞTERİNİN HESAPLARININ LİSTELENMESİ
    BankaService.musteriHesapListele(musteri);
    System.out.print("Para Yatırılacak Hesabı Seçiniz: ");
    int hesapSecimiParaEkleme = input.nextInt();
    //HESAP SAYISINDAN FAZLA VEYA NEGATİF BİR SEÇİM YAPILIRSA HATA MESAJI YAZDIRMA
    if (hesapSecimiParaEkleme > muster.getHesaplar().size() || hesapSecimiParaEkleme < 0) {
        System.out.println("Böyle Bir Hesabınız Bulunmamaktadır!");
    }
    else {
        //HESABIN SEÇİLMESİ
        BankaHesabi secilenHesapParaEkleme = muster.getHesaplar().get(hesapSecimiParaEkleme - 1);

        System.out.print("Hesaba Yatırılacak Para Miktarını Giriniz: ");
        double yatırilanMiktar = input.nextDouble();

        if (yatırilanMiktar >= 0) {
            BankaService.hesabaParaEkle(muster, secilenHesapParaEkleme, yatırilanMiktar);
        }
        else {
            System.out.println("Hesabınıza Böyle Bir Para Yatırılamaz!");
        }
    }

    satirBosluguEkleme();
    break;
```

```
Yapılacak İşlemi Seçin: 2
1) Iban: TR87658956195332874517295, Bakiye: 0.0, Hesap Türü: Vadesiz Hesap
2) Iban: TR11449752144681515529544, Bakiye: 0.0, Hesap Türü: Yatırım Hesabı
Para Yatırılacak Hesabı Seçiniz: 1
Hesaba Yatırılacak Para Miktarını Giriniz: 1500
```

Para yatırılma işlemi sonrası hesap bilgileri:

```
Yapılacak İşlemi Seçin: 1
Müşteri Numarası: 4733278967 Halit
Iban: TR87658956195332874517295, Bakiye: 1500.0, Hesap Türü: Vadesiz Hesap
Iban: TR11449752144681515529544, Bakiye: 0.0, Hesap Türü: Yatırım Hesabı
Kart Numarası: 798336937877454, Limit: 15000.0, Güncel Borç: 1453.0, Kullanılabilir Limit: 13547.0
```

Para yatırılma işleminden sonra kullanıcının yapabileceği bir sonraki işlem para çekme işlemidir. Burada kullanıcıdan aynı para yatırma işleminde olduğu gibi para çekmek istediği banka hesabı istenir ve daha sonrasında çekmek istediği para miktarı sorulmaktadır. Hesabında böyle bir paranın olup olmadığı gibi hata kontrollerinin sağlanması sonucu para çekme işlemi gerçekleştirilmektedir.

Müşterinin hesabından para çekme işleminin kaynak kodu:

```
case 3:
    //MÜŞTERİNİN PARA ÇEKEBİLMESİ İÇİN HESAPLARININ LİSTELENMESİ
    BankaService.musteriHesapListele(musteri);
    System.out.print("Para Çekilecek Hesabı Seçiniz: ");
    int hesapSecimiParaCekme = input.nextInt();

    //İSTENMEYEN GİRİŞ DURUMLARI İÇİN HATA KONTROLÜ
    if (hesapSecimiParaCekme > musterihesaplar.size() || hesapSecimiParaCekme < 0) {
        System.out.println("Böyle Bir Hesabınız Bulunmamaktadır!");
    }
    else {
        BankaHesabi secilenHesapParaCekme = musterihesaplar.get(hesapSecimiParaCekme - 1);

        System.out.print("Hesaptan Çekilecek Para Miktarını Giriniz: ");
        double cekilenMiktar = input.nextDouble();

        if (cekilenMiktar > secilenHesapParaCekme.getBakiye()) {
            System.out.println("Hesabınızda Bu Miktarda Para Bulunmamaktadır!");
        }
        else {
            BankaService.hesaptanParaCek(musteri, secilenHesapParaCekme, cekilenMiktar);
        }
    }

    satirBosluguEkleme();
    break;
```



```
Yapılacak İşlemi Seçin: 3
1) Iban: TR87658956195332874517295, Bakiye: 1500.0, Hesap Türü: Vadesiz Hesap
2) Iban: TR11449752144681515529544, Bakiye: 0.0, Hesap Türü: Yatırım Hesabı
Para Çekilecek Hesabı Seçiniz: 1
Hesaptan Çekilecek Para Miktarını Giriniz: 500
```

Para çekme işlemi sonrası hesap bilgilerinin yazdırılması:

```
Yapılacak İşlemi Seçin: 1
Müşteri Numarası: 4733278967 Halit
Iban: TR87658956195332874517295, Bakiye: 1000.0, Hesap Türü: Vadesiz Hesap
Iban: TR11449752144681515529544, Bakiye: 0.0, Hesap Türü: Yatırım Hesabı
Kart Numarası: 798336937877454, Limit: 15000.0, Güncel Borç: 1453.0, Kullanılabilir Limit: 13547.0
```

Para çekme işleminden sonra ise kullanıcının bir sonraki yapabileceği işlem hesaplar arası para transferidir. Bu işlem sırasında kullanıcıdan 2 adet banka hesabı seçmesi, ve aktarılacak para miktarını girmesi istenmektedir. İstenen girdiler kurallara uygun ve mevcut banka hesapları üzerinden yapıldığında para transferi gerçekleşmektedir.

Müşterinin hesaplar arası para transferi işleminin kaynak kodu:

```
case 4:
    BankaService.musteriHesapListele(musteri);
    //PARANIN GÖNDERİLECEĞİ VE ALINACAĞI HESAPLAR İÇİN SEÇİM ALANI
    System.out.print("Parayı Gönderen Hesabı Seçiniz: ");
    int gonderenHesap = input.nextInt();
    System.out.print("Paranın Yatırılacağı Hesabı Seçiniz: ");
    int yatırilanHesap = input.nextInt();
    if (gonderenHesap > muster.getHesaplar().size() || gonderenHesap < 0) {
        System.out.println("Böyle Bir Hesabınız Bulunmamaktadır!");
    }
    else if (yatırilanHesap > muster.getHesaplar().size() || yatırilanHesap < 0) {
        System.out.println("Böyle Bir Hesabınız Bulunmamaktadır!");
    }
    else {
        //BANKA HESAPLARININ İNDEX'E DİKKAT EDİLEREK BİR NESNEYE ATANMASI
        BankaHesabi gonderen = muster.getHesaplar().get(gonderenHesap - 1);
        BankaHesabi yatırilan = muster.getHesaplar().get(yatırilanHesap - 1);
        System.out.print("Aktarılacak Para Miktarını Giriniz: ");
        double aktarilacakMiktar = input.nextDouble();

        BankaService.paraTransferi(yatırilan, gonderen, aktarilacakMiktar, muster);
    }
    satirBosluguEkleme();
    break;
```



```
Yapılacak İşlemi Seçin: 4
1) Iban: TR87658956195332874517295, Bakiye: 1000.0, Hesap Türü: Vadesiz Hesap
2) Iban: TR11449752144681515529544, Bakiye: 0.0, Hesap Türü: Yatırım Hesabı
Parayı Gönderen Hesabı Seçiniz: 1
Paranın Yatırılacağı Hesabı Seçiniz: 2
Aktarılacak Para Miktarını Giriniz: 300
Para Transferi Başarıyla Gerçekleşti
```

Para transferi sonrası hesap bilgilerinin yazdırılması:

```
Yapılacak İşlemi Seçin: 1
Müşteri Numarası: 4733278967 Halit
Iban: TR87658956195332874517295, Bakiye: 700.0, Hesap Türü: Vadesiz Hesap
Iban: TR11449752144681515529544, Bakiye: 300.0, Hesap Türü: Yatırım Hesabı
Kart Numarası: 798336937877454, Limit: 15000.0, Güncel Borç: 1453.0, Kullanılabilir Limit: 13547.0
```

Para transferinden sonra kullanıcının yapabileceği son işlem ise kredi kartı borcu ödeme işlemidir. Bu işlem sırasında üç adet durum söz konusudur. İlk olarak kullanıcı borcundan fazla bir miktarda borç ödemesi yapmaya çalışabilir. Bu durumda kullanıcının borcu tamamen sıfırlanır ve yatırdığı ekstra para hesabına geri iade edilir. Bir sonraki durum ise borcunun bir kısmını ödemesi olacaktır. Bu durumda toplam borcundan ödediği miktar düşürülecektir ve güncel borcu ekrana yazdırılacaktır. Son durum ise borcun olmadığı halde borç ödemesi yapılma durumudur. Bu durumda kullanıcıya borcunun olmadığına dair bir mesaj döndürülmektedir.

Müşterinin kredi kartı borç ödeme işleminin kaynak kodu:

```
case 5:
    BankaService.musteriKrediKartilistele(musteri);
    System.out.print("Borç Ödemesi Yapılacak Kartı Seçiniz: ");
    int kartIndex = input.nextInt();

    if (kartIndex > musterikrediKartlari().size() || kartIndex < 0) {
        System.out.println("Böyle Bir Kartınız Bulunmamaktadır!");
    }
    else {
        //ALINAN KARTIN İNDEKSİ NUMARASINA GÖRE BİR KART NESNESİNİN OLUŞTURULMASI
        KrediKarti secilenKart = musterikrediKartlari().get(kartIndex - 1);
        System.out.println(secilenKart.toString());
        System.out.print("Ödemek İstedığınız Borç Miktarını Giriniz: ");
        double odenecekBorç = input.nextDouble();

        BankaService.musteriHesapListele(musteri);
        System.out.print("Ödemenin Yapılacağı Hesabı Seçiniz: ");
        int hesapIndex = input.nextInt();
        if (odenecekBorç < 0) {
            System.out.println("Geçersiz Borç Ödeme İşlemi!");
        }
        else if (hesapIndex > musterihesaplar().size() || hesapIndex < 0) {
            System.out.println("Geçersiz Banka Hesabı Seçimi!");
        }
        else {
            BankaService.krediKartiBorçOdeme(secilenKart, odenecekBorç, musterikrediKartlari(), hesapIndex - 1);
        }
    }
    satirBosluguEkleme();
    break;
```

BankaService.krediKartiBorcOdeme() metodunun kaynak kodu:

```
public static void krediKartiBorcOdeme(KrediKarti kart, double odenecekBorc, Musteri muster, int hesapIndex) {
    ArrayList<BankaHesabi> hesaplar = muster.getHesaplar();
    double guncelBorc = kart.getGuncelBorc();
    BankaHesabi odemeYapilacakHesap = hesaplar.get(hesapIndex);
    double hesapBakiye = odemeYapilacakHesap.getBakiye();
    if (guncelBorc == 0) {
        System.out.println("Kartınızda Borç Bulunmamaktadır!");
    }
    else if (odenecekBorc > hesapBakiye) {
        System.out.println("Hesabınızdaki Para Miktarı Bu Ödeme İçin Yetersiz!");
    }
    else if (odenecekBorc > guncelBorc) {
        System.out.println("Borcunuzdan Fazla Ödeme Miktarı Yaptınız Borcunuz Sıfırlanıyor...");
        System.out.println("Kalan Paranız Hesabınıza Geri Yatırılıyor...");
        kart.setGuncelBorc(0);
        odemeYapilacakHesap.setBakiye(hesapBakiye - guncelBorc);
        kart.setKullanilabilirLimit(kart.getKullanilabilirLimit() + guncelBorc);
    }
    else {
        System.out.println("Borç Ödemesi Yapılıyor..");
        kart.setGuncelBorc(guncelBorc - odenecekBorc);
        odemeYapilacakHesap.setBakiye(hesapBakiye - odenecekBorc);
        kart.setKullanilabilirLimit(kart.getKullanilabilirLimit() + odenecekBorc);
        System.out.println("Güncel Borcunuz: "+ kart.getGuncelBorc());
    }
}
```

Yapılacak İşlemi Seçin: 5
1) Kart Numarası: 798336937877454, Limit: 15000.0, Güncel Borç: 1453.0, Kullanılabilir Limit: 13547.0
Borç Ödemesi Yapılacak Kartı Seçiniz: 1
Kart Numarası: 798336937877454, Limit: 15000.0, Güncel Borç: 1453.0, Kullanılabilir Limit: 13547.0
Ödemek İstedığınız Borç Miktarını Giriniz: 2000
1) Iban: TR87658956195332874517295, Bakiye: 2000.0, Hesap Türü: Vadesiz Hesap
2) Iban: TR11449752144681515529544, Bakiye: 300.0, Hesap Türü: Yatırım Hesabı
Ödemenin Yapılacağı Hesabı Seçiniz: 1
Borcunuzdan Fazla Ödeme Miktarı Yaptınız Borcunuz Sıfırlanıyor...
Kalan Paranız Hesabınıza Geri Yatırılıyor...

Fazla para ödenmesi durumunda oluşan geri ödeme:

Yapılacak İşlemi Seçin: 1
Müşteri Numarası: 4733278967 Halit
Iban: TR87658956195332874517295, Bakiye: 547.0, Hesap Türü: Vadesiz Hesap
Iban: TR11449752144681515529544, Bakiye: 300.0, Hesap Türü: Yatırım Hesabı
Kart Numarası: 798336937877454, Limit: 15000.0, Güncel Borç: 0.0, Kullanılabilir Limit: 15000.0

PROJENİN EKSİKLERİ VE GELİŞTİRİLEBİLİR YANLARI:

Geliştirilen bu projenin eksik yönleri aşağıdaki gibidir:

- Alınan girdilerin hata kontrolleri sadece değerler üzerinden yapılmaktadır. Fakat bu alınan girdilerin veri türüne dair bir kontrol yapılmamaktadır. Try-Catch blokları kullanılarak bu hata kontrolleri daha detaylı bir şekilde ele alınabilir.
- Projenin başlangıcında da gösterildiği gibi banka personeli sadece kod üzerinden oluşturulmaktadır. Burada bankadaki kayıtlı kişilerin rollerinin yani sıradan personel, müşteri, müdür gibi rollerinin ayarlanmasıyla sadece müdür rolüne sahip kişilerin banka personeli oluşturabilmesi sağlanabilir. Bunun için veri tabanları kullanılması işlemleri daha kolay hale getirecektir.

Geliştirilen bu projenin geliştirilebilir yanları aşağıdaki gibidir:

- Geliştirilen proje konsol üzerinden işlemlerini gerçekleştirebilmektedir. Bu, kullanıcılar için görsel olarak oldukça zor bir duruma sebep olmaktadır. Bunun yerine proje bir arayüze sahip olarak geliştirilirse daha kullanıcı dostu bir uygulama olabilmektedir.
- Müşteri ve banka personeli tarafından yapılan işlemler kalıcı olmamaktadır. Proje her başlatıldığında ilk haline geri dönerek her işlem tekrar yapılmaktadır. Bunun yerine proje, bir veri tabanına veya bir dosyaya bağlanarak işlemlerin kalıcı olması sağlanabilmektedir. Bu sayede her seferinde yeni kullanıcılar ve banka hesapları üretmektense daha önce oluşturulmuş kullanıcılar ve banka hesapları tekrar tekrar kullanılabilir.